Two implementations of the same MPM

`default` against `warp`, line by line, with the tensors each one allocates

1. What this is

The four MLS-MPM operators — `mpm_strain`, `mpm_scatter`, `mpm_grid_update`, `mpm_gather` — now exist in two forms that a spec selects by name on the `implementation` axis:

- `default` — the prior MPM. Pure PyTorch, dimension-generic (2D and 3D from one body), runs on CPU and on any CUDA device, and **differentiable in use, not merely in principle**: `engine.run(..., grad=True)` keeps the autograd tape across the rollout (`engine.py:1498`) and `discovery_cardio_mpm/train.py` calls `loss.backward()` through exactly these four operators. This is the reference: every claim below is measured against it.
- `warp` — the refactor. `mpm_scatter` and `mpm_gather` become one NVIDIA Warp kernel each: one CUDA thread per particle, and the particle's whole 27-node loop runs inside that thread with the running sums held in registers (§4). 3D and CUDA only. `mpm_strain` and `mpm_grid_update` are unchanged and still run the `default` bodies.

They are the same mechanism and the same equations, so they sit on the `implementation` axis rather than the `model` axis: a spec that swaps one for the other is asserting that the biology did not change. That assertion is what §6 tests.

The headline. On 945 000 particles, seven substeps per frame, one RTX A6000: **1150.8 ms/frame** for `default` against **79.4 ms/frame** for `warp` (**14.5×**), and **52.2 ms/frame** with the substep loop additionally captured as a CUDA graph (**22.0×**). Peak allocated memory falls from 2.94 GiB to 0.35 GiB, which is the reason for the speedup and not a side effect of it.

2. The cycle, and what each operator owes

MLS-MPM carries the material on particles p and solves on a background grid i that is thrown away every substep. Per substep, with Δx the cell size, w_{ip} the quadratic B-spline weight of node i at particle p, and $\mathbf{x}_{ip} = \mathbf{x}_i - \mathbf{x}_p$:

$$\textbf{strain} \quad \mathbf{F}_p \leftarrow (\mathbf{I} + \Delta t\, \mathbf{C}_p)\, \mathbf{F}_p \tag{1}$$

$$\textbf{scatter} \quad m_i = \sum_p w_{ip} m_p, \qquad (m\mathbf{v})_i = \sum_p w_{ip}\big[m_p \mathbf{v}_p + \mathbf{A}_p\, \mathbf{x}_{ip}\big] \tag{2}$$

$$\mathbf{A}_p = -\tfrac{4\Delta t}{\Delta x^2} V_p^0\, \boldsymbol{\sigma}_p + m_p \mathbf{C}_p, \qquad \boldsymbol{\sigma}_p = 2\mu(\mathbf{F}_p - \mathbf{R}_p)\mathbf{F}_p^\top + \lambda J(J-1)\mathbf{I} \tag{3}$$

$$\textbf{grid} \quad \mathbf{v}_i = (m\mathbf{v})_i / \max(m_i, \epsilon), \quad \text{then gravity, walls, plates, surface tension} \tag{4}$$

$$\textbf{gather} \quad \mathbf{v}_p = \sum_i w_{ip} \mathbf{v}_i, \qquad \mathbf{C}_p = \tfrac{4}{\Delta x} \sum_i w_{ip}\, \mathbf{v}_i \otimes (\mathbf{o}_i - \mathbf{f}_p), \qquad \mathbf{x}_p \leftarrow \mathbf{x}_p + \Delta t\, \mathbf{v}_p \tag{5}$$

$\mathbf{F}_p$ is the **deformation gradient** — one $D \times D$ matrix per particle (3×3 in 3D, so `p.F` is $[N, 3, 3]$, initialised to the identity at `entities.py:238`). It is the Jacobian $\partial\mathbf{x}/\partial\mathbf{X}$ of the map from a particle's reference position to its current one, so it carries the accumulated stretch, shear and rotation since $t = 0$; strain and stress are derived *from* it, and it is not itself a strain tensor. $\mathbf{R}_p$ is its orthogonal polar factor (the rotation part) and $J = \det \mathbf{F}_p$ is the volume ratio, 1 for undeformed material; $\mathbf{o}_i \in \{0, 1, 2\}^3$ is the node's offset within the stencil and $\mathbf{f}_p$ the particle's fractional cell position, so $\mathbf{o}_i - \mathbf{f}_p$ is $\mathbf{x}_{ip}$ in cell units. **The stencil is $3^3 = 27$ nodes, not $3^2 = 9$**: the quadratic B-spline has support of three cells *per axis*, and in three dimensions the tensor product of three axes is 27. Every per-particle cost in this note carries that factor of 27.

The contract both implementations sign. `mpm_scatter` reads `pos, vel, C, F, mass, mu, la, pvol` and the parent's acceleration, and accumulates into `mpm_grid.m` and `.mv` *in place*. It must **not** touch `F` — an earlier attempt fused the strain update into the scatter kernel for free, and advanced `F` twice per substep; the grid mass still agreed to eleven digits while `F` disagreed at 3.3×10^{-4}. `mpm_gather` reads `mpm_grid.v`, writes `pos`, `vel` and `C` in place, and leaves dormant particles (`occ`$= 0$, the growth reservoir) exactly where they were.

3. `default`, line by line: what each line allocates

The gather is the shorter of the two and makes the point completely. This is `mpm_ops.py` lines 790–845, with the tensors folded in; N is the particle count, $S = 27$, $D = 3$.

```
offsets = stencil_offsets(D, dev); S = offsets.shape[0] # [27,3] memoised, free
X, V = p.get("pos"), p.get("vel") # views free
fx, weight, flat = bspline(X, inv_dx, offsets, g.shape, periodic)
    base = (X * inv_dx - 0.5).floor().long() # [N,3] i64 22.7 MB
    fx = X * inv_dx - base.float() # [N,3] f32 11.3 MB
    w = stack(3 quadratic terms) # [N,3,3] f32 34.0 MB
    weight = ones(N,S); for k: weight *= w[:, oidx[:,k], k] # [N,27] f32 102.1 MB x6
    gpos = base[:,None,:] + oidx[None] # [N,27,3] i64 612.4 MB
    comps = [gpos[...,k].clamp(0, shape[k]-1) for k in 0..2] # [N,27] i64 204.1 MB x3
    flat = ((c0 * ny) + c1) * nz + c2 # [N,27] i64 204.1 MB x4
gvn = g.v[flat].view(p.n, S, D) # [N,27,3] f32 306.2 MB
new_V = (weight[..., None] * gvn).sum(1) # [N,27,3] f32 306.2 MB
dpos_grid = offsets[None] - fx[:, None, :] # [N,27,3] f32 306.2 MB
new_C = 4*inv_dx*(weight[...,None,None] *
       (gvn[...,:,None] @ dpos_grid[...,None,:])).sum(1) # [N,27,3,3] f32 918.5 MB x2
...
```

The last line is the whole story. `gvn[...,:,None] @ dpos_grid[...,None,:]` is $N \times 27$ independent $3{\times}1$ by $1{\times}3$ outer products; PyTorch has no way to express "form this $3{\times}3$, weight it, add it to a running sum, discard it". It must **materialise all $N \times 27$ of them**, write **918.5 MB** to global memory, read it back to multiply by `weight`, write **another 918.5 MB**, read *that* back to reduce over the stencil axis, and only then produce the $[N, 3, 3]$ that is actually wanted — **34.0 MB**. The useful output is **1.8%** of the traffic spent producing it.

3.1 The full bill, measured

The listing above is read off the source; the tables below are *measured*. A `TorchDispatchMode` records every `aten` call one real substep makes and the shape and dtype of every tensor it allocates — including the ones the source never names: the broadcast temporaries, the `index` gathers hiding behind `w[:, oidx[:,k], k]`, the int64 index arithmetic. Views (`expand`, `unsqueeze`, `view`, `select`) are **excluded** — they return a new tensor object over the same storage and move nothing, and counting them would inflate `default`'s bill with bytes it never touches. Rows below 40 MB are omitted; the totals include everything. Regenerate with `python tools/mpm_warp_note.py -trace`.

`mpm_gather`, `default` ($N = 945\,000$, one substep)

aten op	shape	dtype	each	×	total
bmm	[25515000, 3, 3]	float32	918.5 MB	1	918.5 MB
mul	[945000, 27, 3, 3]	float32	918.5 MB	1	918.5 MB
add	[945000, 27, 3]	int64	612.4 MB	1	612.4 MB
clamp	[945000, 27]	int64	204.1 MB	3	612.4 MB
mul	[945000, 27]	int64	204.1 MB	2	408.2 MB
add	[945000, 27]	int64	204.1 MB	2	408.2 MB
index	[945000, 27]	float32	102.1 MB	3	306.2 MB
mul	[945000, 27]	float32	102.1 MB	3	306.2 MB
index	[25515000, 3]	float32	306.2 MB	1	306.2 MB
mul	[945000, 27, 3]	float32	306.2 MB	1	306.2 MB
sub	[945000, 27, 3]	float32	306.2 MB	1	306.2 MB
ones	[945000, 27]	float32	102.1 MB	1	102.1 MB
mul	[945000, 3]	float32	11.3 MB	6	68.0 MB
sub	[945000, 3]	float32	11.3 MB	4	45.4 MB
all allocating ops					**6060 MB**
per particle					**6413 B**
÷ essential (864 B)					**7.4**×

`mpm_scatter`, `default` (same substep)

aten op	shape	dtype	each	×	total
clone	[945000, 27, 3, 3]	float32	918.5 MB	1	918.5 MB
mul	[945000, 27]	float32	102.1 MB	6	612.4 MB
add	[945000, 27, 3]	int64	612.4 MB	1	612.4 MB
clamp	[945000, 27]	int64	204.1 MB	3	612.4 MB
mul	[945000, 27, 3]	float32	306.2 MB	2	612.4 MB
mul	[945000, 27]	int64	204.1 MB	2	408.2 MB
add	[945000, 27]	int64	204.1 MB	2	408.2 MB
index	[945000, 27]	float32	102.1 MB	3	306.2 MB
sub	[945000, 27, 3]	float32	306.2 MB	1	306.2 MB
bmm	[25515000, 3, 1]	float32	306.2 MB	1	306.2 MB
add	[945000, 27, 3]	float32	306.2 MB	1	306.2 MB
index_select	[945000, 3, 3]	float32	34.0 MB	8	272.2 MB
mul	[945000, 3, 3]	float32	34.0 MB	8	272.2 MB
add	[945000, 3, 3]	float32	34.0 MB	6	204.1 MB
linalg_cross	[945000, 3, 3]	float32	34.0 MB	4	136.1 MB
div	[945000, 3, 3]	float32	34.0 MB	4	136.1 MB
mul	[945000, 3]	float32	11.3 MB	10	113.4 MB
ones	[945000, 27]	float32	102.1 MB	1	102.1 MB
sub	[945000, 3]	float32	11.3 MB	5	56.7 MB
all allocating ops					**7177 MB**
per particle					**7594 B**
÷ essential (864 B)					**8.8**×

Per particle per substep, the four `default` operators allocate **313 B** (strain) + **7594 B** (scatter) + **151 B** (grid) + **6413 B** (gather). The *essential* traffic — what any correct MLS-MPM must move, and the constant `tools/mpm_bench.py` divides by — is 216 floats = 864 B: read `pos/vel/C/F/mass/mu/la`, scatter mass and momentum to 27 nodes, gather velocity back from 27. So `default` moves **16.7**× more bytes than the algorithm requires, and the two $[N, 27, 3, 3]$ tensors are most of it.

4. `warp`, line by line: the same arithmetic, nowhere to put it

The Warp kernel is one thread per particle. Everything the table above lists as a tensor is, here, a local variable — and a local variable in a CUDA thread is a register, which is not memory.

Nothing is ever 27 wide, and that is the point. It would be wrong to picture 27 nodes held at once: a thread has ≈64 registers to work with, nowhere near enough for 27 copies of (weight, node velocity, offset). The loop instead *visits* the 27 nodes one at a time and **accumulates**. Only

two things persist across iterations — `newv` (3 floats) and `newC` (9 floats), twelve registers of running sum — and each node's weight, velocity and offset live for exactly one iteration before the next node overwrites them. The register cost is therefore $O(1)$ in the stencil size, not $O(27)$, which is precisely what PyTorch cannot express: having no way to say "form this, add it to a running sum, discard it", it must make the loop a tensor axis and materialise all 27 slices at once.

`ptxas -arch=sm_86 -O3 -verbose` on Warp's emitted PTX confirms the arithmetic worked out: **64 registers, 0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads** for both `g2p` and `p2g_atomic`. **Zero spill is the load-bearing number** — had the working set exceeded the register file, the compiler would have quietly pushed the overflow to local memory, which is global memory wearing a different name, and the fusion would have bought nothing.

```
@wp.kernel # src/plexus/operators/mpm_warp.py:193
def g2p(X, STATE, C, GV, OCC, LIQ, pa, va, ngx, ngy, ngz, dx, dt, ...):
    p = wp.tid() # one thread per particle
    x = X[p] # vec3 -> 3 registers
    base = wp.vec3(floor(x[k]*inv_dx - 0.5) for k) # vec3 -> 3 registers
    fx = x*inv_dx - base # vec3 -> 3 registers
    newv = wp.vec3(0,0,0) # vec3 -> 3 registers
    newC = wp.mat33(0,...,0) # mat33 -> 9 registers
    for i in range(3): # the 27-node stencil, UNROLLED by the compiler
      wi = <quadratic in fx[0]> # the [N,3,3] 'w' tensor: 1 register, reused
      for j in range(3):
        wj = <quadratic in fx[1]>
        for k in range(3):
          wk = <quadratic in fx[2]>
          w = wi*wj*wk # the [N,27] 'weight' tensor: 1 register
          gv = GV[(gi*ngy + gj)*ngz + gk] # the [N,27,3] 'gvn' tensor: 3 registers
          dpos = wp.vec3(i-fx[0], j-fx[1], k-fx[2]) # the [N,27,3] 'dpos_grid': 3 registers
          newv = newv + w*gv # the [N,27,3] product: never exists
          newC = newC + (4*inv_dx*w)*wp.outer(gv, dpos) # the [N,27,3,3]: NEVER EXISTS
    ...
    STATE[p, pa+0..2] = xn ; STATE[p, va+0..2] = newv ; C[p] = newC # the only writes
```

The correspondence is exact, term for term. What changes is where the intermediate lives:

quantity	`default` tensor	at $N = 945\,000$	`warp`
B-spline weights per axis	`[N,3,3]` f32	34.0 MB	3 registers, recomputed per tap
stencil weight	`[N,27]` f32	102.1 MB ×6	1 register
node indices	`[N,27,3]` i64	612.4 MB	3 integers, computed inline
flattened index	`[N,27]` i64	204.1 MB ×4	1 integer
gathered node velocity	`[N,27,3]` f32	306.2 MB	`wp.vec3`, 3 registers
$\mathbf{x}_{ip}$ in cells	`[N,27,3]` f32	306.2 MB	`wp.vec3`, 3 registers
outer product $\mathbf{v}_i \otimes \mathbf{x}_{ip}$	`[N,27,3,3]` f32	**918.5 MB** ×2	**accumulated into 9 registers**
allocated per substep, both fused operators		**13 237 MB**	**0 MB**

The traced `warp` run confirms it: `mpm_scatter` and `mpm_gather` do not appear in the allocating-op trace *at all*. What remains is `mpm_strain` (313 B/particle) and `mpm_grid_update` (151 B/particle), both still `default` bodies, for a total of **0.5**× the essential traffic against `default`'s **16.7**×.

The scatter differs in one way the gather does not. Grid→particle is a pure read: no two threads write the same place, so there is nothing to coordinate. Particle→ grid is the opposite — many particles land on the same node, and the kernel uses `wp.atomic_add`, 108 of them per particle (27 nodes × (1 mass + 3 momentum)). That is why the two fusions do not cost the same and do not buy the same (§5), and it is the remaining wall (§7).

Differentiability is not lost in principle, only unwired here. Warp emits a backward kernel next to every forward one — `g2p_..._cuda_kernel_backward` sits in the same PTX file — so the reason `warp` carries `DIFFERENTIABLE = False` is not that the tool cannot do it. It is that this port launches with a plain `wp.launch` outside a `wp.Tape`, and `wp.from_torch` wraps the tensors without `requires_grad`, so no backward is registered with autograd. Wiring it up is a separate job; until it is done, anything that needs a gradient — `discovery_cardio_mpm`'s training loop above all — must

stay on `default`, and that is a second reason (alongside byte-identity, §6) why `default` is not going away.

One convenience worth naming. `wp.mat33` is a first-class type with `determinant`, `inverse`, `transpose` and `outer`, so the polar iteration $\mathbf{R} \leftarrow \frac{1}{2}(\mathbf{R} + \mathbf{R}^{-\top})$ is three lines. The same kernel written in Triton needed twenty-seven hand-rolled cofactor expressions, because Triton has no matrix type. Fewer lines is not the argument; *fewer places to put a cofactor sign error* is.

5. What it buys

5.1 Which fusion did what

Each operator was switched on its own, on the 945 000-particle benchmark, RTX A6000, 12 timed frames after 4 warm-up frames.

`mpm_scatter`	`mpm_gather`	ms/frame	peak	speedup
default	default	1150.8	2.94 GiB	1.0×
warp	default	466.9	2.80 GiB	2.5×
default	**warp**	767.8	2.94 GiB	1.5×
warp	**warp**	79.4	0.35 GiB	14.5×
warp	**warp** +capture	52.2	0.55 GiB	22.0×

The two fusions are superlinear together. Alone they are worth 2.5× and 1.5× on the frame; together they are worth 14.5×, and $2.5 \times 1.5 = 3.75$, not 14.5. The reason is in the memory column, which only moves when *both* land: 2.94 GiB → 2.80 → 2.94 → **0.35**. Fusing one end still leaves the other end's 918.5 MB temporary alive, so the allocator's high-water mark is unchanged, the caching allocator keeps thrashing, and every other kernel in the substep keeps paying for it. The last 918.5 MB is not one seventh of the problem — it is the problem.

5.2 Scaling: 20 million particles now run

RTX A6000 (48 GiB, peak 768 GB/s), `warp`, capture off, 12 timed frames:

particles	sub	ms/frame	ms/substep	GB/s	% peak
945 000	7	77.8	11.12	73.4	9.6%
1 999 998	7	161.8	23.12	74.7	9.7%
4 999 995	7	402.1	57.44	75.2	9.8%
9 999 990	7	784.9	112.13	77.1	10.0%
20 000 007	7	1614.1	230.58	74.9	9.8%
100 000 008	7	7968.9	1138.42	75.9	9.9%

Throughput is **flat at ≈75 GB/s across a 106× range in problem size** — 73.4 GB/s at 945 k and 75.9 GB/s at 100 M — and memory is linear at **0.309 GiB per million particles**. There is no knee anywhere: the cost per particle at 100 M is the cost per particle at 945 k.

The target the exercise was set for — 20 k cells × 1000 particles = 20 M — fits on one A6000 at 1.61 s/frame and 6.23 GiB, with room for five times more. One hundred million particles run on the same 48 GiB card at 7.97 s/frame and 30.9 GiB. The comparison that makes that number legible: on the A100, `default` at **10 M** needed **30.5 GiB** and **11.60 s/frame**. `warp` therefore fits **ten times the particles in the same memory**, and a frame of 100 M costs **less** than a frame of 10 M did.

Under `default` 20 M does not run at all — not on this card and, as §5.3 measures, not on an 80 GiB A100 either.

5.3 A100 baseline

NVIDIA A100-SXM4-80GB (peak 1555 GB/s), `warp`, both capture settings:

	capture off			**capture on**		
particles	ms/frame	GB/s	GiB	ms/frame	GB/s	GiB
500 013	40.7	74.2	0.22	**24.0**	126.2	0.37
999 999	71.3	84.9	0.37	**46.7**	129.6	0.58
4 999 995	378.5	79.9	1.61	**229.8**	131.6	2.29
9 999 990	778.6	77.7	3.14	**459.5**	131.6	4.42
20 000 007	1544.8	78.3	6.23	**913.2**	132.5	8.68
100 000 008	7853.9	77.0	30.89	**4551.0**	132.9	42.81

And the same card running `default`, which is where the sweep stops:

particles	sub	ms/frame	ms/substep	GB/s	% peak
500 013	7	581.3	83.05	5.2	0.3%
999 999	7	1158.3	165.47	5.2	0.3%
4 999 995	7	5790.6	827.23	5.2	0.3%
9 999 990	7	11598.0	1656.86	5.2	0.3%

`default` does not reach 20 M on an 80 GiB card. It dies asking for **18.11 GiB in a single allocation** at 20 M and **60.35 GiB** at 100 M — those are individual temporaries, not totals. `warp` runs 100 M on a 48 GiB A6000.

The most informative row in this note is a `default` row. On the A100, `default` runs at **5.2 GB/s**; on the A6000 it runs at **5.0 GB/s**. A card with twice the memory bandwidth buys the prior implementation **4%**. That is the proof that `default` was never bandwidth-bound: it was bound by the count of kernel launches and by allocator work, neither of which a faster card improves. Fusion is not an optimisation of the old design; it is the only thing that could have helped.

And `warp` is not bandwidth-bound either. At 100 M with capture off it runs at 7853.9 ms on the A100 against 7968.9 ms on the A6000 — **1.5% for twice the bandwidth**. Both cards land at ≈77 GB/s, which is ≈5% of A100 peak and ≈10% of A6000 peak: the *same absolute* throughput on very different hardware, which is the signature of a serialised bottleneck rather than a memory one. That bottleneck is the scatter's atomics (§7).

What CUDA-graph capture actually removes, and it is not launch latency. Capture is worth **1.73×** at 100 M (7853.9 → 4551.0 ms/frame) — but at 4.5 s a frame, the 28 kernel launches it elides are nanoseconds against seconds, so launch overhead cannot be the mechanism. The memory column names it: peak rises **30.89 → 42.81 GiB** under capture, a 39% increase. A captured graph allocates from a private pool that stays resident, so `mpm_strain`'s and `mpm_grid_update`'s $[N, 3, 3]$ intermediates are allocated *once* instead of being malloc'd and freed 7 times a frame. Capture is buying the removal of the caching allocator, which is the same thing fusion bought — and it buys it for the two operators fusion has not reached yet.

6. Precision

`warp` is **not** byte-identical to `default` and cannot be, for two independent reasons: the 27-tap sums are reassociated (the kernel accumulates sequentially, PyTorch's `.sum(1)` reduces pairwise), and `wp.atomic_add` in the scatter commits in whatever order the hardware schedules. So this is an `implementation` with a tolerance gate, not a promotion twin with `tools/mpm_identity_gate.py`.

Measured after 2 frames × 7 substeps on 108 k particles, both implementations run from the same seed:

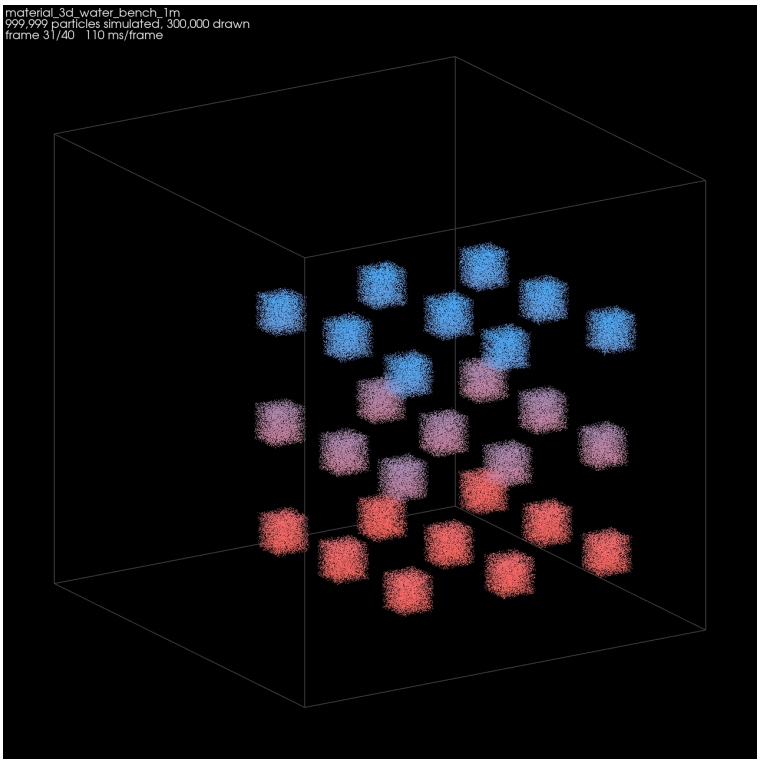

Figure 1: One frame of `material_3d_water_bench_1m` rendered by `tools/mpm_live_movie.py`: 999 999 particles simulated, 300 000 of them drawn, colour fixed at $t = 0$ by height so that mixing stays visible. **The picture is taken off the GPU while the run proceeds and no trajectory is written** — at 100 M a single recorded frame is 1.2 GB, so a 60-frame clip would need 72 GB on disk before a renderer saw a pixel. The tool is separate from `tools/mpm_bench.py`, which renders nothing: the `ms/frame` stamped on the movie includes the device→host copy and the render and is slower than the benchmark's by construction.

switched	quantity	max $\lvert\Delta\rvert$	max relative	reference scale
gather only	`pos`	1.79×10^{-7}	4.3×10^{-7}	$\lvert x\rvert \approx 0.40$
gather only	`F`	8.34×10^{-7}	8.3×10^{-7}	$\lvert F\rvert \approx 0.33$
gather only	grid `m`	3.07×10^{-12}	—	total mass identical to 6 digits
both	`pos`	1.79×10^{-7}	5.3×10^{-7}	$\lvert x\rvert \approx 0.40$
both	`F`	8.34×10^{-7}	8.3×10^{-7}	$\lvert F\rvert \approx 0.33$
both	grid `mv`	1.46×10^{-11}	—	$\lvert mv\rvert \approx 1.8 \times 10^{-8}$

Float32 has $\epsilon = 1.19 \times 10^{-7}$, so a position of 0.40 has an ulp of 2.98×10^{-8} and the observed 1.79×10^{-7} is **6 ulp after 14 substeps** — round-off, accumulated at the rate reassociation predicts. The large *relative* figures on the grid (up to 1.2×10^{-4}) are on nodes whose mass is $\approx 10^{-8}$: a tiny denominator, not a disagreement. Total grid mass matches to six digits.

What this costs, stated plainly. A run cannot be reproduced bit-for-bit across the two implementations, so the `implementation` field is part of a run's identity and must be recorded with it. Anything that needs byte-identity — the promotion twins — stays on `default`.

7. What is left

1. `mpm_strain` **and** `mpm_grid_update` **are still** `default` and are now the whole remaining frame: 313 B and 151 B per particle-substep against `warp`'s zero for the other two. `strain` is a $[N, 3, 3]$ matrix product and fuses trivially; `grid_update` is $O(\text{cells})$ and is a different kernel shape.

2. **The atomics are the wall.** `warp` sits at ≈8–10% of peak on both cards and does not improve when the card gets faster, which is the same signature `default` showed for a different reason. The fix is the node-centric P2G that NVIDIA's own `warp.fem` MPM uses: sort particles by cell into CSR, one thread *per grid node*, each summing its own contributions with a plain `+=` — zero atomics. The read amplification that argument was rejected on earlier (27×) is a coalesced streaming read, not the same kind of cost as a serialised atomic write.

3. **Memory is no longer the constraint, and that is new.** At 0.309 GiB per million particles an 80 GiB A100 holds ≈250 M and a 48 GiB A6000 ≈150 M. Past ≈20 M the fixed 96^3 grid of this benchmark stops being a discretisation anyone would choose, so the next question is not how many particles fit but what grid they need — and a finer grid costs substeps through the CFL bound $\Delta t \leq \mathrm{cfl} \cdot \Delta x / c$, which is a different scaling problem from this one.

Provenance. Benchmarks: `tools/mpm_bench.py`, tables regenerated by `tools/mpm_warp_note.py`. Specs: `material_3d_water_bench_500k` through `_200m` under `config/material/` — 27 blobs of radius 0.05 in a unit box, 96^3 grid, $\Delta t = 3.6 \times 10^{-3}$ in 7 substeps, *held fixed across the sweep* so the only variable is particle count. A production 20 M run would use a finer grid and correspondingly more substeps; this sweep deliberately does not, because a benchmark that changes two things at once measures neither.